Homework-III

Nishan Khanal

```r
library(tidyverse)
library(tidymodels)
library(ggformula)
library(GGally)
library(ISLR)
library(skimr)
library(car)
```

Backward selection

Use the Auto dataset (can be found in ISLR R package). Our objectives:

- Predict acceleration using the remaining quantitative variables.
- Which, if any, of the variables are significant predictors of acceleration.
- How good is the resulting prediction?

1. Remove the name and origin columns from the dataset. Then
   - Produce a ggpairs plot. Write a short discussion (2–3 sentences) about what the plot suggests for multiple linear regression (collinearity, pairwise relationships, etc.).
2. Start from the full model with all quantitative predictors. Do a manual backward selection using p-values to determine what to remove until all predictors are significant at the $= 0.05$ level. Filter the tidy() output to show only candidates for removal, i.e. p-values $>$. Remove the predictor with the largest p-value at each step. Compare the full (unfiltered) tidy() results and adjusted R2 at each step to the previous step.
3. With the final model (from (2)), check residuals, interpret coefficients, report confidence intervals on coefficients.

Load the auto data and skim it to know more about data dimension, data types and completion rate.

```r
data(Auto, package="ISLR")
glimpse(Auto)
```

```
## Rows: 392
## Columns: 9
## $ mpg          <dbl> 18, 15, 18, 16, 17, 15, 14, 14, 14, 15, 15, 14, 15, 14, 2~
## $ cylinders    <dbl> 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 4, 6, 6, 6, 4, ~
## $ displacement <dbl> 307, 350, 318, 304, 302, 429, 454, 440, 455, 390, 383, 34~
## $ horsepower   <dbl> 130, 165, 150, 150, 140, 198, 220, 215, 225, 190, 170, 16~
## $ weight       <dbl> 3504, 3693, 3436, 3433, 3449, 4341, 4354, 4312, 4425, 385~
## $ acceleration <dbl> 12.0, 11.5, 11.0, 12.0, 10.5, 10.0, 9.0, 8.5, 10.0, 8.5, ~
## $ year         <dbl> 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 7~
## $ origin       <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 1, 1, 1, 3, ~
## $ name         <fct> chevrolet chevelle malibu, buick skylark 320, plymouth sa~
```

Let's know more about the different variables in this dataset:

`mpg`: miles per gallon

`cylinders`: Number of cylinders between 4 and 8

`displacement`: Engine displacement (cu. inches)

`horsepower`: Engine horsepower

`weight`: Vehicle weight (lbs.)

`acceleration`: Time to accelerate from 0 to 60 mph (sec.)

`year`: Model year (modulo 100)

`origin`: Origin of car (1. American, 2. European, 3. Japanese)

`name`: Vehicle name

Discard the name and origin column and skim the dataset

```
auto <- Auto |>
  dplyr::select(-name, -origin)

skim(auto)
```

Table 1: Data summary

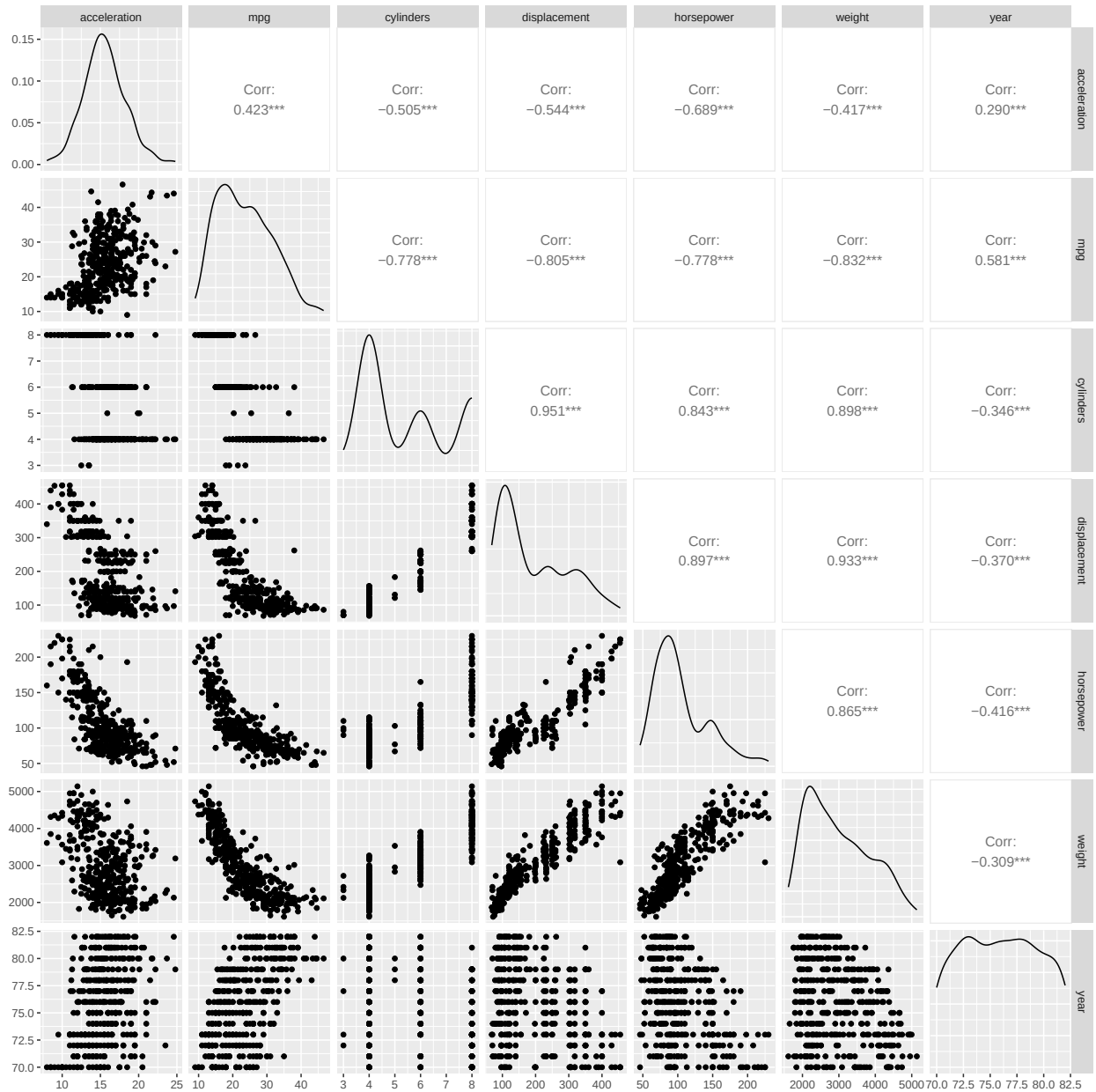
Name	auto
Number of rows	392
Number of columns	7
Column type frequency:	
numeric	7
Group variables	None

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
mpg	0	1	23.45	7.81	9	17.00	22.75	29.00	46.6	
cylinders	0	1	5.47	1.71	3	4.00	4.00	8.00	8.0	
displacement	0	1	194.41	104.64	68	105.00	151.00	275.75	455.0	
horsepower	0	1	104.47	38.49	46	75.00	93.50	126.00	230.0	
weight	0	1	2977.58	849.40	1613	2225.25	2803.50	3614.75	5140.0	
acceleration	0	1	15.54	2.76	8	13.78	15.50	17.02	24.8	
year	0	1	75.98	3.68	70	73.00	76.00	79.00	82.0	

The average time in seconds to go from 0 to 60 mph represented as `acceleration` for the vehicles in this dataset is 15.54s, and the average mpg is 23.45.

```r
auto |>
  select(acceleration, everything()) |>
  ggpairs()
```



`horsepower` has the highest absolute correlation with our response variable(acceleration) closely followed by displacement and cylinders, and then mpg. year has weak positive correlation.

We can also notice our predictors are highly correlated with each other. For example, the correlation between cylinders and displacement is 0.951 - very strong. Other strong correlation include between displacement and weight, cylinders and horsepower, cylinders and weight, displacement and horsepower, displacement and mpg, horsepower and weight, and weight and mpg. This can cause issue when fitting the model due to multicollinearity.

Question 2

Start from the full model with all quantitative predictors. Do a manual backward selection using p-values to determine what to remove until all predictors are significant at the = 0.05 level. Filter the tidy() output to show only candidates for removal, i.e. p-values > . Remove the predictor with the largest p-value at each step. Compare the full (unfiltered) tidy() results and adjusted R2 at each step to the previous step.

Since we will be fitting a model multiple times with backward elimination, and report the output, let's create a reusable function to report the output

```
report_step <- function(model, step_title="Step 1", alpha = 0.05) {
  td  <- tidy(model)
  gl  <- glance(model)


  # candidates for removal: p.value > alpha (exclude intercept)
  candidates <- td |>
    filter(term != "(Intercept)", p.value > alpha) |>
    arrange(desc(p.value))


  print(step_title)
  cat("\n\nFull Tidy output\n")
  print(td)
  cat("\n\n")

  cat("Glance into the model\n")
  print(gl)
  cat("\n\n")

  print("Candidates to be removed:")
  print(candidates)

  invisible(list(model = model, tidy = td, glance = gl, candidates = candidates))
}

# create a specification for a linear regression model

lm_spec <- linear_reg() |>
  set_mode("regression") |>
  set_engine("lm")
```

Let's split our data into train and test set so that we can evaluate our model later.

```
set.seed(1)

# put 80% of the data into the train set and 20% in test set
auto_split <- initial_split(auto, prop = 0.80)

auto_train <- training(auto_split)
auto_test <- testing(auto_split)
```

Step 1: Full model (all the predictors)

Let's start building the model by including all the predictors.

```
alpha = 0.05

formula_1 <- acceleration ~ mpg + cylinders + displacement + horsepower + weight + year

model_1 <- lm_spec |>
  fit(formula_1, data = auto_train)

output_1 = report_step(model_1, step_title = "Step 1: Full (all predictors)", alpha=0.05)
```

```
## [1] "Step 1: Full (all predictors)"
##
##
## Full Tidy output
## # A tibble: 7 x 5
##   term         estimate std.error statistic  p.value
##   <chr>           <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)  18.8      2.46        7.64   2.72e-13
## 2 mpg           0.0300   0.0283      1.06   2.90e- 1
## 3 cylinders     0.00934  0.193       0.0484 9.61e- 1
## 4 displacement -0.0110   0.00424    -2.59   1.01e- 2
## 5 horsepower   -0.0837   0.00619   -13.5    5.25e-33
## 6 weight        0.00335  0.000375    8.92   4.49e-17
## 7 year         -0.0405   0.0367     -1.11   2.69e- 1
##
##
## Glance into the model
## # A tibble: 1 x 12
##   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
##       <dbl>         <dbl> <dbl>     <dbl>    <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1     0.584         0.576  1.75      71.6 2.22e-55     6  -616. 1249. 1279.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
##
##
## [1] "Candidates to be removed:"
## # A tibble: 3 x 5
##   term      estimate std.error statistic p.value
##   <chr>        <dbl>     <dbl>     <dbl>   <dbl>
## 1 cylinders  0.00934    0.193     0.0484   0.961
## 2 mpg        0.0300     0.0283    1.06     0.290
## 3 year      -0.0405     0.0367   -1.11     0.269
```

Looking at the full tidy output, we see that `horsepower`, and `weight` are highly significant based on the p-values. Additionally, displacement is also significant. The relationship between the predictors and the response seem to follow the same relationship as we had seen in the correlation analysis.

This model with all the predictors explains 57.58% of the variance in `acceleration`.

Looking at the variables with coefficients whose p.value > 0.05, we see there are three such predictors, namely: cylinders, mpg and year - in order of decreasing p-value. Let's remove cylinders (variable whose coefficient has the highest p-value).

Step 2 : Removing `cylinders`

```r
formula_2 <- acceleration ~ mpg + displacement + horsepower + weight + year

model_2 <- lm_spec |>
  fit(formula_2, data = auto_train)

output_2 = report_step(model_2, step_title = "Step 2: Removed `cylinders`")
```

```
## [1] "Step 2: Removed `cylinders`"
##
##
## Full Tidy output
## # A tibble: 6 x 5
##   term         estimate std.error statistic  p.value
##   <chr>           <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)  18.8      2.43          7.77 1.20e-13
## 2 mpg           0.0300   0.0282        1.06 2.89e- 1
## 3 displacement -0.0108   0.00300      -3.60 3.68e- 4
## 4 horsepower   -0.0837   0.00617     -13.6  3.27e-33
## 5 weight        0.00335  0.000374      8.96 3.18e-17
## 6 year         -0.0405   0.0366       -1.11 2.69e- 1
##
##
## Glance into the model
## # A tibble: 1 x 12
##   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
##       <dbl>         <dbl> <dbl>     <dbl>    <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1     0.584         0.577  1.75      86.2 2.27e-56     5  -616. 1247. 1273.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
##
##
## [1] "Candidates to be removed:"
## # A tibble: 2 x 5
##   term  estimate std.error statistic p.value
##   <chr>    <dbl>     <dbl>     <dbl>   <dbl>
## 1 mpg     0.0300    0.0282      1.06   0.289
## 2 year   -0.0405    0.0366     -1.11   0.269
```

Looking at the full tidy output, we see that `horsepower`, `weight`, and `displacement` are highly significant based on the p-values. Removing statistically insignificant predictors changed the p-value of `displacement` significantly.

This model without `cylinders` explains the 57.71% of the variance in `acceleration`, similar to the earlier one.

Looking at the variables with coefficients whose p.value > 0.05, we now see there are two such predictors: `mpg` and `year`. Let's remove mpg (variable whose coefficient has the highest p-value).

Step 3 : Removing `mpg`

```
formula_3 <- acceleration ~ displacement + horsepower + weight + year

model_3 <- lm_spec |>
  fit(formula_3, data = auto_train)

output_3 = report_step(model_3, step_title = "Step 3: Removed `mpg`")
```

```
## [1] "Step 3: Removed `mpg`"
##
##
## Full Tidy output
## # A tibble: 5 x 5
##   term         estimate std.error statistic  p.value
##   <chr>           <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)  18.4      2.39         7.70  1.88e-13
## 2 displacement -0.0109   0.00300     -3.61  3.52e- 4
## 3 horsepower   -0.0841   0.00615    -13.7   1.43e-33
## 4 weight        0.00317  0.000331     9.55  4.22e-19
## 5 year         -0.0176   0.0296      -0.596 5.52e- 1
##
##
## Glance into the model
## # A tibble: 1 x 12
##   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
##       <dbl>         <dbl> <dbl>     <dbl>    <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1     0.582         0.577  1.75      107. 3.59e-57     4  -617. 1246. 1268.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
##
##
## [1] "Candidates to be removed:"
## # A tibble: 1 x 5
##   term  estimate std.error statistic p.value
##   <chr>    <dbl>     <dbl>     <dbl>   <dbl>
## 1 year   -0.0176    0.0296    -0.596   0.552
```

Looking at the full tidy output, we still see that `horsepower`, `weight` and `displacement` are highly significant based on the p-values. The relationship between the predictors and the response also follows the same relationship as we had seen in the correlation analysis.

This model without `cylinders` and `mpg` explains 57.70% of the variance in `acceleration`.

Looking at the variables with coefficients whose p.value > 0.05, we now one such predictor - year.

Step 4 : Removing `year`

```
formula_4 <- acceleration ~ displacement + horsepower + weight
```

```r
model_4 <- lm_spec |>
  fit(formula_4, data = auto_train)

output_4 = report_step(model_4, step_title = "Step 4: Removed `year`")
```

```
## [1] "Step 4: Removed `year`"
##
##
## Full Tidy output
## # A tibble: 4 x 5
##   term         estimate std.error statistic  p.value
##   <chr>           <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)  17.0      0.548        31.0  5.66e-97
## 2 displacement -0.0107   0.00299      -3.58 3.92e- 4
## 3 horsepower   -0.0832   0.00597     -14.0  1.21e-34
## 4 weight        0.00314  0.000329      9.56 3.90e-19
##
##
## Glance into the model
## # A tibble: 1 x 12
##   r.squared adj.r.squared sigma statistic  p.value    df logLik   AIC   BIC
##       <dbl>         <dbl> <dbl>     <dbl>    <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1     0.582         0.578  1.75      143. 3.29e-58     3  -617. 1244. 1263.
## # i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
##
##
## [1] "Candidates to be removed:"
## # A tibble: 0 x 5
## # i 5 variables: term <chr>, estimate <dbl>, std.error <dbl>, statistic <dbl>,
## #   p.value <dbl>
```

Looking at the full tidy output, we see that `horsepower`, `weight`, and `displacement` are the significant predictors of `acceleration`. These predictors are obtained through the backward elimination process.

This model without `mpg`, `cylinders`, `year` explains 57.79% of the variance in `acceleration`, similar to the model above.

Since we know that these predictors are correlated with each other. Let's calculate VIF to quantify their severity of multicollinearity

```r
model_4$fit |>
  vif()
```

```
## displacement   horsepower       weight
##     9.369602     4.852330     7.611251
```

Since two variables have VIF greater than 5, it indicates that the model might be unstable and we should be cautious while making inference on the model. However, since their regression coefficients follow the same relationship to the target as we had seen during the correlation analysis, I've decided to keep them as the predictors for the final model.

Question 3

With the final model (from (2)), check residuals, interpret coefficients, report confidence intervals on coefficients.

The final model can be expressed as:

$$\widehat{acceleration} = \beta_0 + \beta_1 \times displacement + \beta_2 \times horsepower + \beta_3 \times weight$$

Substituting the coefficients:

$$\widehat{acceleration} = 17.0136 - 0.0107 \times displacement - 0.0832 \times horsepower + 0.0031 \times weight$$

Coefficient interpretation

- β_0: It does not make sense to interpret this value because this represents the time it takes for a vehicle to go from 0 to 60mph whose displacement, horsepower and weight is 0 (Not possible).
- β_1: Keeping everything else constant, increasing the displacement of the engine by 1 cu inch reduces the predicted time it takes to go from 0 to 60 (`acceleration`) by 0.0107 second. However, it is difficult to keep other variables constant when increasing displacement, because it has a strong correlation with weight and horsepower.
- β_2: Keeping everything else constant, increasing the horsepower of the vehicle by 1 unit reduces the predicted time it takes to go from 0 to 60(`acceleration`) by 0.0832 second. However, it is difficult to keep other variables constant when increasing horsepower, because it has a strong correlation with displacement and weight.
- β_3: Keeping everything else constant, increasing the weight of the vehicle by 1 lb increases the predicted time it takes to go from 0 to 60(`acceleration`) by 0.0031 second. However, it is difficult to keep other variables constant when increasing weight, because it has a strong correlation with displacement and horsepower.

Confidence interval of coefficeints

```r
tidy(model_4, conf.int = TRUE)
```

```
## # A tibble: 4 x 7
##   term         estimate std.error statistic  p.value conf.low conf.high
##   <chr>           <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
## 1 (Intercept)  17.0      0.548        31.0  5.66e-97 15.9      18.1
## 2 displacement -0.0107   0.00299      -3.58 3.92e- 4 -0.0166   -0.00484
## 3 horsepower   -0.0832   0.00597     -14.0  1.21e-34 -0.0950   -0.0715
## 4 weight        0.00314  0.000329      9.56 3.90e-19  0.00249   0.00379
```

- Confidence interval of β_0: It does not make sense to interpret this value because this represents the time it takes for a vehicle to go from 0 to 60mph whose displacement, horsepower and weight is 0 (Not possible).
- Confidence interval of β_1: Keeping everything else constant, we are 95% confident that increasing the displacement of the engine by 1 cu inch reduces the predicted time it takes to go from 0 to 60 (`acceleration`) by between 0.0166 and 0.0048 seconds. However, it is difficult to keep other variables constant when increasing displacement, because it has a strong correlation with weight and horsepower.

- Confidence interval of β_2: Keeping everything else constant, increasing the horsepower of the vehicle by 1 unit reduces the predicted time it takes to go from 0 to 60(`acceleration`) by between 0.0950 and 0.0715 seconds. However, it is difficult to keep other variables constant when increasing horsepower, because it has a strong correlation with displacement and weight.
- Confidence interval of β_3: Keeping everything else constant, increasing the weight of the vehicle by 1 lb increases the predicted time it takes to go from 0 to 60(`acceleration`) by between 0.0025 and 0.0038 seconds. However, it is difficult to keep other variables constant when increasing weight, because it has a strong correlation with displacement and horsepower.

Model Assessment

```
auto_test |>
  mutate(.pred = predict(model_4 ,new_data=auto_test)$.pred) |>
  rsq(truth=acceleration, estimate=.pred)
```

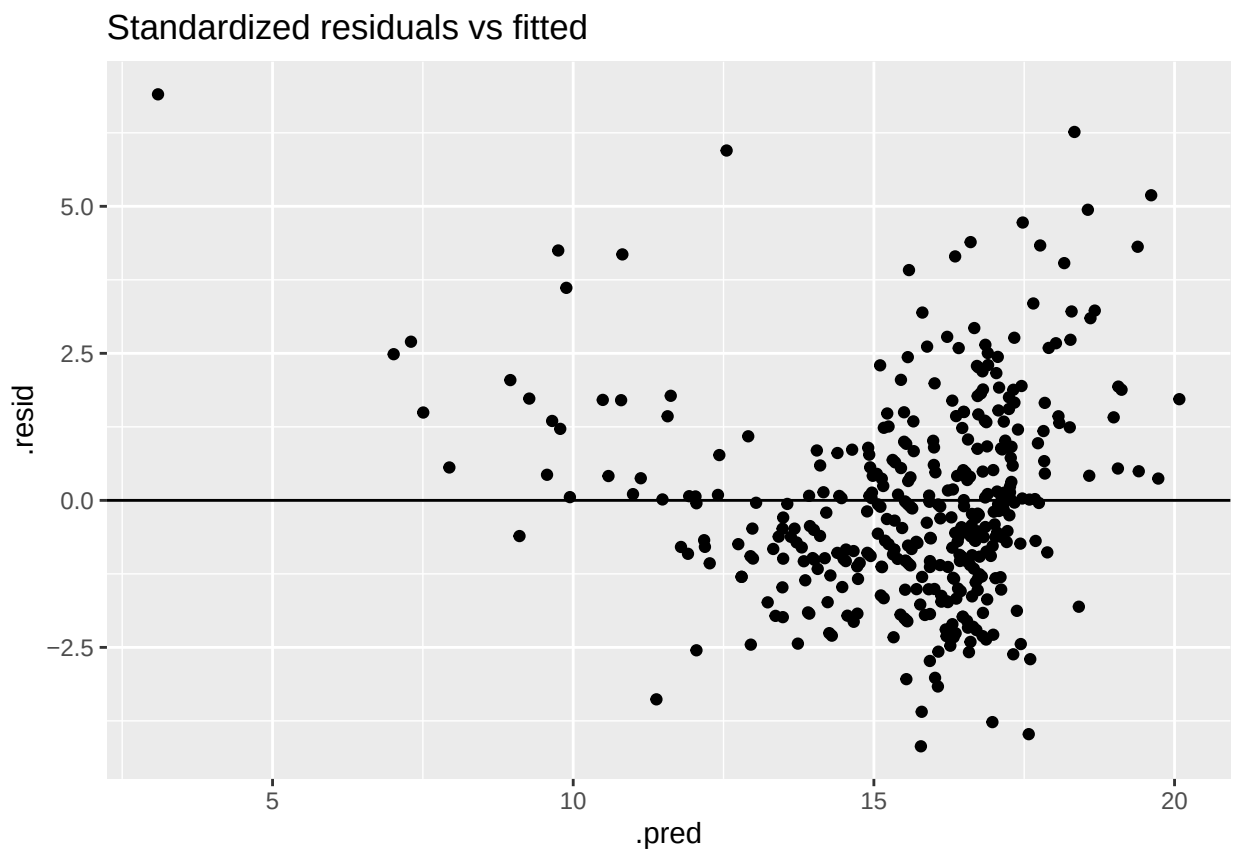
```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rsq     standard       0.722
```

The predictors in the final model explain 72% of the variance in the acceleration in the test set. So, we can say that the model generalized well for unseen datasets.

Assumptions analysis Since we fitted the linear regression model which has the following assumptions, let's test them for:

- Linearity
- Homoscedasticity
- Normality of Residuals

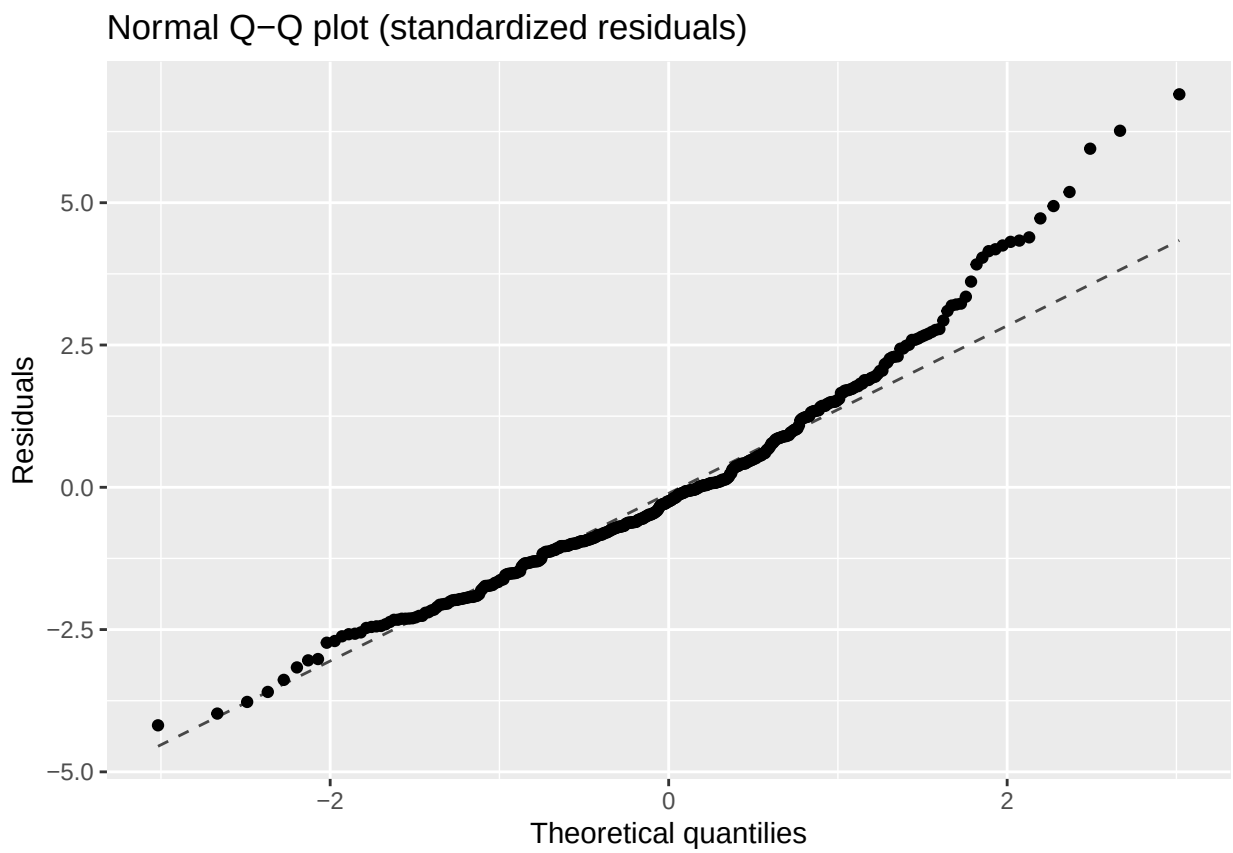
```
auto_aug <- augment(model_4, new_data = auto)

gf_point(.resid ~ .pred, data = auto_aug) |>
  gf_hline(yintercept = 0) |>
  gf_labs(title = "Standardized residuals vs fitted")
```



It seems like the there's a bow shaped pattern, which suggest the relationship between the predictors and the acceleration may be non-linear. Additionally, the spread of the residual first decreases and then increases again, which suggests heteroscedasticity.

To check the whether or not the residuals follow normal distribution let's draw a qq plot.

```
gf_qq(~.resid, data = auto_aug) |>
  gf_qqline() |>
  gf_labs(title = "Normal Q-Q plot (standardized residuals)",
          x = "Theoretical quantilies",
          y = 'Residuals')
```



While most points in the middle lie on the line, however, the a lot of points at the top stray away from the line, so the residual may not follow normal distribution.

Conclusion

Based, on the residual analysis, we can't certainly say the assumptions of normality, linearity between the predictors and the response, and heteroscedasticity of residuals are satisfied well. Additionally, it should also be noted that the predictors are highly correlated with each other. So, we should also be cautious while doing inference.